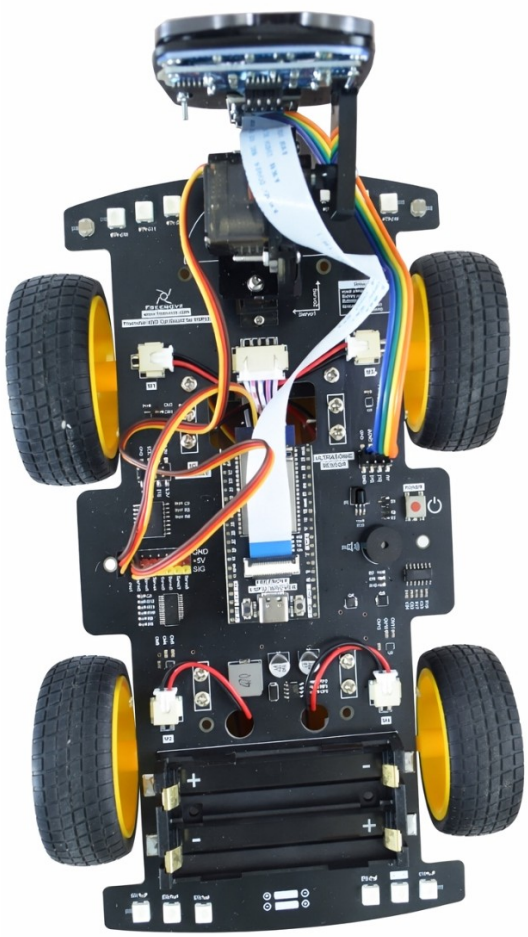
Werner-von-Siemens-Schule Mannheim

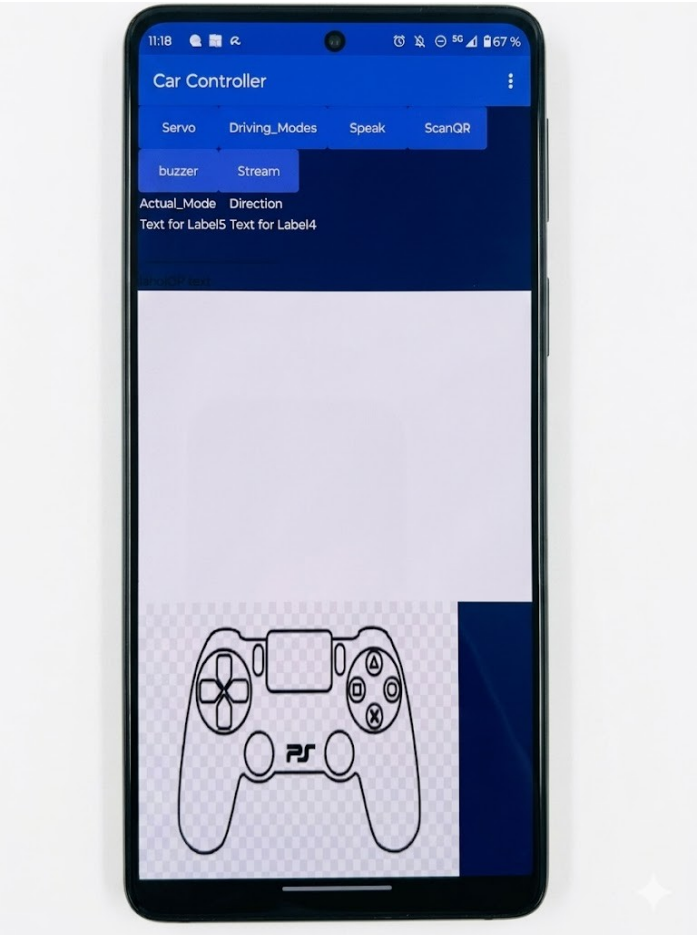
“WiFi-Controlled ESP32 Robotic Vehicle with camera and ultrasonic sensor”

Project-supervisor : Herr Spitzer

Name : Cyrille Tatse

Class : 2BKI21





Inhaltsverzeichnis

1 Introduction....5
1.1 Abstract....5
1.2 Project Planning and Organization....5
1.3 Materials and Components....6
Core Hardware (Freenove 4WD Kit)....6
1.4 Software & Development Tools....7
2 Component Functions and Technical Roles....7
2.1 Hardware Components....8
2.2 Software Tools....9
3 Assembling of the car....9
3.1 Assembling of the car’s base foundation....9
3.2 Assembling of the servos....10
Camera and Ultrasonic sensor mounting....11
4 Mobile app development (MIT App Inventor)....11
4.1 App base structure....12
4.2 listpicker selection....12
4.3 Essential Application Components....12
5 Control modes....12
5.1 Joystick Mode....12
5.2 Tilt mode....13
5.3 Voice control....14
5.4 QR Code mode....14
6 Servo and Camera Control....15
6.1 Multi-Purpose Joystick Logic....15
7 Precision and Smoothness....16
7.1 Real-Time Visual Feedback....16
8 Arduino code implementation....17
8.1 Initialization(Setup())....17
8.2 Distance Measurement (getDistance())....17
8.3 WiFi access point (softAP())....18
8.4 Dual-port Web server....18
8.5 Movement and servo control....18
8.6 Safety systems....19
9 Problems and solution....19
9.1 Bluetooth bandwidth limitations....20
9.2 ESP32 single host limitation....20
9.3 Servo misalignment after mounting....20
9.4 Joystick center point miscalculation....20
9.5 Tilt mode interfering with other modes....21
9.6 Camera Failure Due to insufficient PSRAM....21
10 Testing....21
11 Conclusion....22
12 How to Use the WiFi-Controlled ESP32 Vehicle....23
12.1 Switch Between Car and Servo Control (Joystick Mode only)....24
13 Sources....24
13.1 App and Arduino codes....25
14 English Summary....25
15 AI Usage....26

Declaration of Authorship

I hereby declare that I have completed this project work independently and used no other sources or aids than those cited.

All passages that are taken verbatim or in spirit from other works have been identified as such by citing the sources. All passages taken verbatim have been marked as quotations.

Mannheim, Date: **28.04.2026** Name :**Cyrille Tatse**

Signature : _____

Design and Implementation of a WiFi-Controlled ESP32 Vehicle with Camera and Sensors

1 Introduction

1.1 Abstract

This project documents the design and implementation of a WiFi-controlled robotic vehicle built on the Freenove 4WD ESP32 platform. The system integrates an ESP32 microcontroller, a pan-tilt camera module, an ultrasonic distance sensor, and four DC motors Into a single cohesive platform. A custom Android application developed with MIT App Inventor provides the user interface, offering four distinct control modes: joystick, tilt, voice, and QR code. Communication between the app and the vehicle is handled entirely over WiFi, with the ESP32 hosting a dual-port web server that separates the camera stream from the movement commands. The documentation covers the full development process, including hardware assembly, embedded C++ programming, mobile app development, and the technical challenges encountered along the way.

The primary objective of this project was to bridge the gap between software development and hardware engineering. To achieve this goal, a custom-controlled vehicle based on the **Freenove 4WD car kit** was developed.

The vehicle is equipped with an **ultrasonic sensor** for obstacle detection and collision avoidance, as well as a **camera module** capable of streaming live video directly to a mobile device. For user-friendly control, the car is operated through a **custom application created with MIT App Inventor**, which offers **four different driving modes** to allow flexible and intuitive navigation.

This project combines mechanical construction, embedded programming, and mobile application development into a single integrated system.

1.2 Project Planning and Organization

To keep the project structured and on schedule, **GanttProject** was used as the main project management tool. This helped ensure professional organization throughout the development process.

GanttProject was used to:

- **Visualize the timeline** by mapping all technical milestones, from hardware assembly to final software integration
- **Manage time efficiently** by monitoring the duration of each development phase

- **Optimize tasks** by focusing on one logical step at a time, reducing errors and avoiding unnecessary multitasking

Task Name	Begin Date	End Date	Duration
1. Planning and Preparation	9/17/25	9/24/25	8 Days
* Project idea developed	9/17/25	9/17/25	1 Day
* Software planning (Arduino etc.)	9/18/25	9/19/25	2 Days
* Research on required components	9/22/25	9/24/25	3 Days
2. App Development	9/24/25	10/8/25	15 Days
* User interface (UI) constructed	9/24/25	9/25/25	2 Days
* Bluetooth client created	9/26/25	9/29/25	4 Days
* Control mode selection programmed	9/30/25	10/8/25	9 Days
3. Hardware Assembly	10/15/25	10/21/25	7 Days
* Mounting the display and camera	10/15/25	10/16/25	2 Days
* Installation of ultrasonic sensor	10/17/25	10/17/25	1 Day
* Assembly of the 4WD chassis	10/20/25	10/21/25	2 Days
4. Miscellaneous / Others	10/23/25	12/3/25	42 Days
5. Project Documentation	12/4/25	2/25/26	84 Days

This structured approach significantly improved efficiency and reduced development mistakes.

1.3 Materials and Components

The project required a combination of hardware components and software tools. These are listed below.

Core Hardware (Freenove 4WD Kit)

Component	Function
ESP32 Microcontroller	Acts as the “brain” of the car; handles processing, WiFi communication, and motor control.
Smart Car Chassis	Mechanical base supporting all electronic and mechanical components.
4 DC Motors & Wheels	Provide four-wheel-drive (4WD) movement and steering

	capabilities.
Ultrasonic Sensor	Detects obstacles and enables automatic collision prevention.
Servo Motors	Control pan/tilt movement for the camera and ultrasonic sensor.
Buzzer	Generates audible alerts and safety warnings during operation.

1.4 Software & Development Tools

- **MIT App Inventor** : It's a block-based environment used to create the custom Android control application
- **GanttProject** : Project management software used for planning and progress tracking
- **Arduino IDE (C++)** : Used to program the ESP32 logic, motor control, servo, and camera server

2 Component Functions and Technical Roles

The following diagram shows how all components (hardware and software) interact so everything can function together

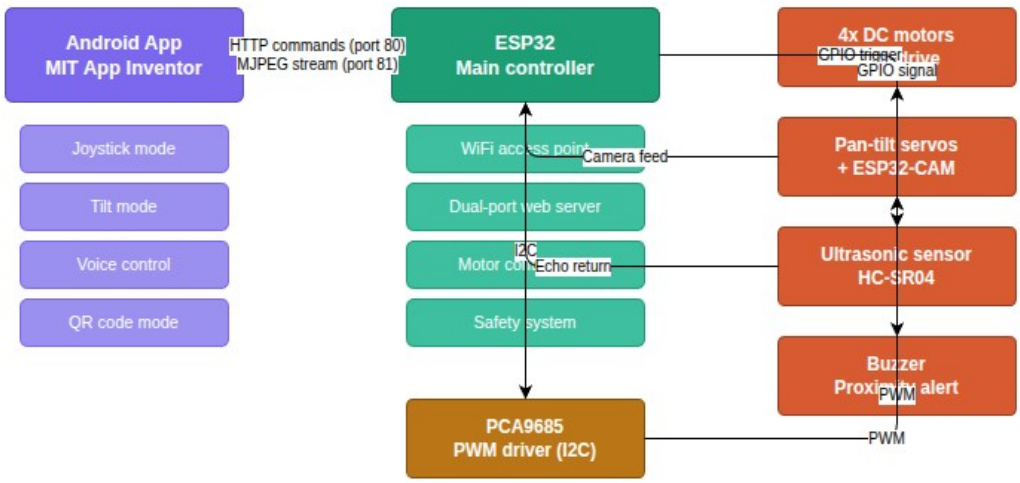


Schaubild 1: Communication Between hardware and software (created with draw.io)

The block diagram above provides a visual overview of the complete system architecture. At the center of the system is the **ESP32 microcontroller**, which acts as the main controller and communication hub for all components.

The **Android app** communicates with the ESP32 exclusively over WiFi. Movement and servo commands are sent as HTTP requests on **port 80**, while the live camera stream is received separately on **port 81**. This dual-port approach ensures that driving the car and viewing the stream never interfere with each other.

On the hardware side, the ESP32 sends motor speed and direction signals to the **PCA9685 PWM driver** via I2C, which then controls the four DC motors. The two **pan-tilt servos** and the **ESP32-CAM** are also managed through the PCA9685, allowing the camera head to move independently of the car's movement.

The **HC-SR04 ultrasonic sensor** continuously sends distance data back to the ESP32 via GPIO. When an obstacle is detected within a critical range, the ESP32 activates the **buzzer** as an acoustic warning.

All communication between the app and the car happens wirelessly the ESP32 operates as its own WiFi access point, meaning no external router or internet connection is required.

2.1 Hardware Components

The hardware represents the physical body and sensory organs of the vehicle.

- **ESP32 Microcontroller:** This is the "brain" of the car. It handles all logical operations, including managing the WiFi Access Point, processing incoming commands from the MIT App, and controlling the motor drivers and camera server.

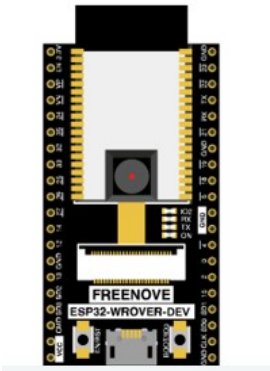


Schaubild 2: Microcontroller(ESP32)

(source: https://docs.freenove.com/projects/fnk0053/en/latest/fnk0053/codes/tutorial/List.html)

- **Smart Car Chassis:** The chassis serves as the structural foundation. It provides the necessary support and mounting points for the motors, battery packs, and electronic circuit boards.
- **4 Motors and Wheels:** These components are responsible for the physical mobility of the car. Using a four-wheel-drive (4WD) configuration allows for high traction and the ability to perform "spot turns" by rotating the left and right sides in opposite directions.
- **Ultrasonic Sensor (HC-SR04):** This sensor acts as the vehicle's primary safety device for detecting and preventing collisions. It works by emitting high-frequency sound waves and

measuring the time it takes for the echo to bounce back, allowing the ESP32 to calculate the distance to an obstacle and make the car perform an action to avoid the collision .

- **Servos:** These are high-precision motors used to provide the camera and ultrasonic sensor with vertical and horizontal movement (Pan/Tilt). This expands the field of view without needing to move the entire car.
- **Buzzer:** The buzzer is used for acoustic signaling. It provides audible warnings when the car is in reverse or when the ultrasonic sensor detects an object that is too close for safety.

2.2 Software Tools

The software provides the project with organization and a user interface.

- **MIT App Inventor:** This platform was used to design and program the custom smartphone application. It converts user inputs (like joystick drags or voice commands) into HTTP requests that are sent wirelessly to the ESP32.
- **GanttProjekt:** This project management tool was essential for tracking the project's evolution. It allowed for a structured timeline, ensuring that hardware assembly, software development, and testing phases were completed in a logical, efficient sequence.
- **Arduino IDE:** This is the primary Integrated Development Environment used to write, compile, and upload the vehicle's firmware. Using the **C++** programming language, I used this tool to program the ESP32 to understand motor commands, process ultrasonic sensor data, and host the web server for the camera stream.
- **Draw.io :** Used for the creation of the block diagram

3 Assembling of the car

3.1 Assembling of the car’s base foundation

I began with the assembly of the vehicle's drivetrain on the underside of the chassis. To achieve optimal stabilization , I began by securing the motor mounting brackets. Precise alignment during this step was vital, as any deviation in the mounting angle would cause the car to pull to one side rather than driving in a straight line during the control.

Once the supports were firmly in place and the chassis felt rigid, I integrated the four motors into their respective brackets. I took care to tighten each screw securely to withstand the mechanical vibrations that occur during active driving, which prevents the hardware from loosening over time. The final step involved attaching the wheels directly to the motor shafts. By ensuring a flush and secure connection between the wheels and the motors, I established a robust physical foundation. This was necessary to ensure that the car could accurately translate the electronic signals from the

ESP32 into smooth, predictable movement across various surfaces without significant errors. And i proceeded by connecting the motorwire to their corresponding terminals (M1-M4)

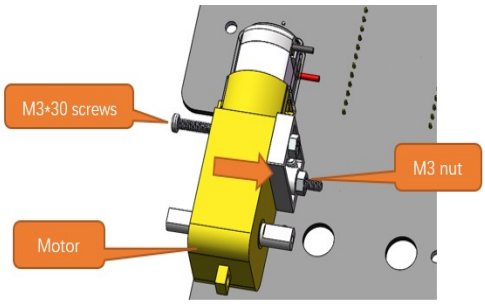


Schaubild 4: Motor fixation

(fixation of the motors)

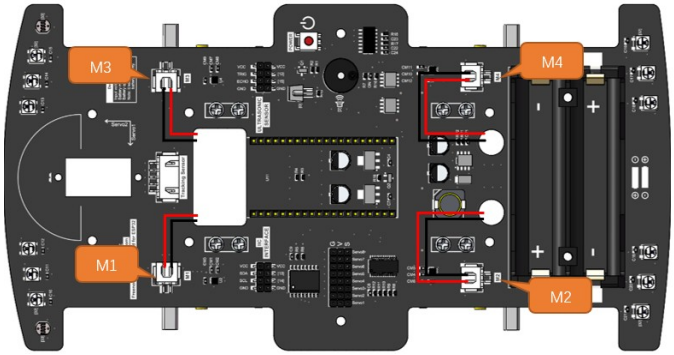


Schaubild 3: motor termiinals

(M1-4 corresponds to the motor terminals)

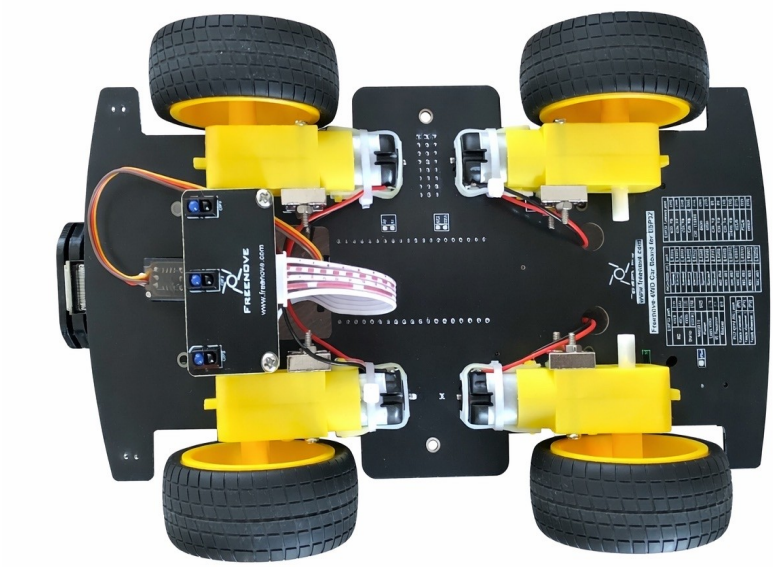


Schaubild 5: Wheels attached to Motors

3.2 Assembling of the servos

With the car's basic structure finished, I moved on to what I refer to as the pan-tilt mechanism. This part of the build involved stacking two servos to dramatically increase the car's range of motion. The idea was to give the camera and ultrasonic sensor both horizontal and vertical movement, essentially creating a neck that allows the car to look around its environment.

This setup meant the car was no longer stuck with a fixed, tunnel-vision perspective. Instead, the camera could scan a wider area, and the ultrasonic sensor could sweep across its surroundings to detect obstacles more effectively. That added flexibility plays a big role in improving navigation and overall safety.

Putting everything together required some careful alignment. I started by mounting the first servo into the base, which controls the side-to-side panning motion. Once it was securely in place, I attached the plastic mounting brackets that connect the two motors. These brackets made it possible to mount the second servo on top of the first, giving it the up-and-down tilting motion. Seeing the full assembly come together was incredibly satisfying at that point, the car evolved from a simple moving platform into something much more interactive, capable of actively scanning its surroundings like a true explorer.

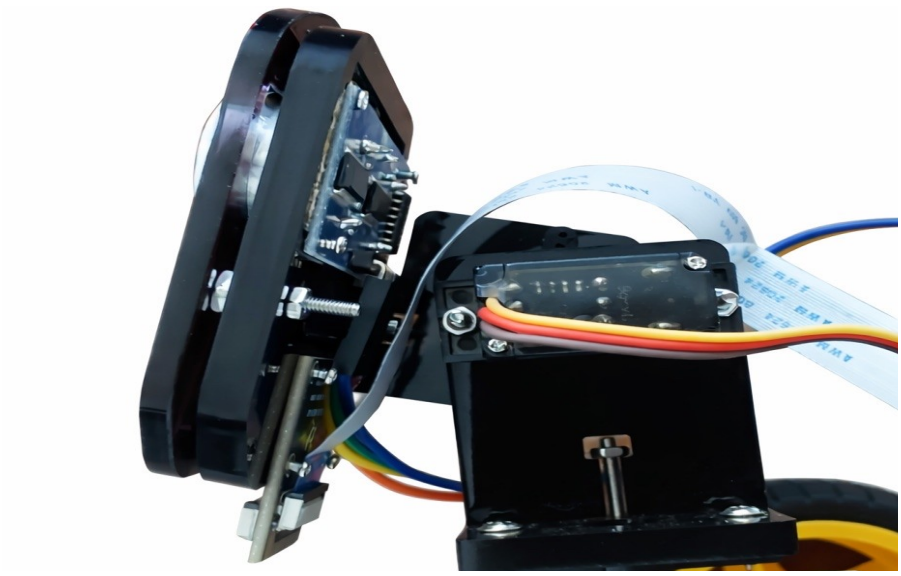


Schaubild 6: Pan-Tilt system

Camera and Ultrasonic sensor mounting

Next, I had to handle the camera's positioning. Since I wanted it to move freely on the servos, the standard short cable just wasn't going to work. To fix this, I swapped it for a longer extension, which gave me the extra length needed to route the wiring safely.

With that extra slack, I was able to mount both the camera and the ultrasonic sensor onto the upper part of the car. Placing them at this higher vantage point made a huge difference; it gave the camera a much better view and allowed the sensor to scan for obstacles with a completely clear path, free from any interference from the car's own chassis.

4 Mobile app development (MIT App Inventor)

For this project my main challenge was to try and make everything by myself and not use a finished app to control the car so I used MIT app inventor to make an app because its a very user-friendly app since its all about drag and drop and it has a very user friendly interface

4.1 App base structure

I started by adding all the necessary components (labels, sensor, clock, WiFi, canvas, bluetooth etc) into the app. Later on I started adding the basic frames adding a few buttons in order to get familiar with the app

4.2 listpicker selection

In order to facilitate the navigation between the different driving modes, A ListPicker component was implemented to allow navigation between different control modes, and by doing things this way it made things way more easier and less confusing.

4.3 Essential Application Components

To build the app's functionality, I integrated several key components that handle everything from data processing to the user interface:

- **Orientation Sensor:** Captures the phone's tilt angles to enable the **Tilt-to-Drive** feature.
- **Clock:** A timer that regulates the speed of data transmission to prevent network lag.
- **Canvas:** Acts as the touch-sensitive area for the Joystick interface.
- **Web1:** The communication bridge that sends HTTP commands to the ESP32 via WiFi.
- **SpeechRecognizer:** Converts voice commands into text strings for the Voice Control mode.
- **Notifier:** Provides the user with pop-up alerts regarding connection status or errors.
- **WebViewer:** A dedicated window that displays the Live Camera Stream within the app.

5 Control modes

I programmed four different modes into the app to test different ways of interacting with the car. Each mode uses the phone's sensors in a unique way to send commands over WiFi to the ESP32.

5.1 Joystick Mode

This is the most natural and reliable way to drive. I placed a virtual joystick on the phone screen that works just like a video game controller. As I move my thumb on the screen, the app calculates the direction and tells the car how to move in real-time.

To make the control professional and responsive, I implemented three specific logic steps in the code:

- **Coordinate Calculation:** I used a **Canvas.Dragged** event to track my finger's position. The app calculates the distance from the center and stores it in variables **(dx and dy)** to determine if the car should move forward, backward, or turn.

- **The "Dead Zone" (Threshold):** I added a threshold variable (set to 30) to create a dead zone in the center. This ensures the car doesn't accidentally twitch or move when my thumb is resting near the middle or if there is slight sensor jitter.
- **Command Filtering:** To keep the WiFi connection fast, I used a **lastCommand** variable. The app checks if the new command is different from the previous one before sending it; this prevents the phone from "flooding" the ESP32 with identical messages, which would otherwise cause lag.
- **Fixing the Center Point Problem** When I first started, the joystick was sending wrong numbers and the car wasn't moving correctly. I realized the app didn't know exactly where the "middle" of my screen was. To fix this, I added a piece of code that runs as soon as the app starts (**Screen1.Initialize**). This code calculates the exact center of the canvas by dividing its width and height by 2. Once I set these centerX and centerY points, the joystick math worked perfectly.

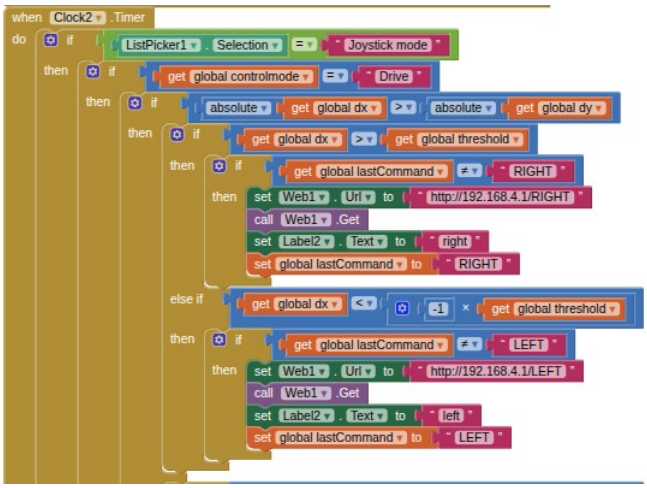


Schaubild 7: Joysticj mode Logic(MIT App Inventor)

5.2 Tilt mode

This mode turns the smartphone itself into a steering wheel by using the phone’s internal orientation sensors. By leaning the phone forward, backward, or side-to-side, I can control the car's direction without touching any buttons.

To make this work accurately, I programmed the following logic into the app's blocks:

- **Axis Mapping:** The app constantly monitors the phone's **Pitch** and **Roll**. I used a "Clock" timer to check these values multiple times a second. Based on my testing, I set specific math conditions to trigger movement:
    - **Forward:** Triggered if the Pitch is less than -20.
    - **Backward:** Triggered if the Pitch is greater than 20.

    - **Right/Left:** These are triggered when the Roll goes beyond 20 or below -20.

- **The "Neutral" Zone:** You'll notice in the code that if the tilt is between -20 and 20 (the "else" block), the app sends a **STOP** command. This creates a balanced center point so the car stays still when you hold the phone level, preventing it from moving when you don't want it to.
- **Real-time Feedback:** Every time a tilt command is sent, I programmed the app to update a label on the screen (e.g., "forward" or "left"). This helped me during testing to make sure the app was "hearing" the sensors correctly before I even looked at the car.

  To make the tilt controls reliable, I had to fix three main issues:

  The specific challenges I encountered with this mode (mode conflicts, sensitivity tuning, and axis remapping) are documented at the bottom(problems and solutions).

5.3 Voice control

This mode allows for hands-free driving by converting speech into movement. To make this work, I used the **SpeechRecognizer** component, which handles the complex task of turning audio into text.

- **Logic and Keyword Matching:** I programmed the app to wait for a "result" from the speech sensor. Once it receives the text, it checks if the result **contains** specific keywords like "forward," "backward," or "stop." If a match is found, the app immediately sends the corresponding HTTP command to the ESP32.
- **Flexible Recognition:** By using the "contains" block instead of an "equal to" block, the app is more forgiving. For example, if I say "Move forward," the app ignores the word "Move" and only focuses on the command "forward."
- **The Latency Challenge:** During testing, I noticed a delay between speaking and the car moving. This is because the audio is processed on Google's servers before the text is sent back to my app. Because of this lag, I added a manual **STOP** command as a fallback to ensure safety.

Due to these limitations, voice control is the least recommended mode for precise driving. The specific challenges like server latency, command misinterpretation, and environmental interference are documented at the bottom

5.4 QR Code mode

I included a QR code scanner as an experimental feature. In this mode, I programmed the car to follow instructions printed on "Command Cards." This allows the vehicle to perform missions based on what it my phone camera scans and sends to the car

Logic and Safety Features:

- **Mode Validation:** As seen in my blocks, I added a safety check. If I press the scan button while in the wrong mode, a **Notifier** pop-up appears saying "choose cards mode first." This prevents the camera from opening accidentally and keeps the app logic clean.
- **Automated Execution:** Once a valid QR code is scanned, the `BarcodeScanner.AfterScan` event automatically triggers the `Web1.Get` request. This means as soon as the camera "sees" the QR code, my phone reads the QR code and copies the text it contains e.g(/FORWARD) in an input field and as soon as the content of the input field is changed, the app sends the input field's content directly. By this way, I could save much time because, I don't have to send the content myself after scanning the QR code. So, the car reacts instantly without me having to press any other buttons or type anything.

  While this mode is the most "high-tech" feeling, it had the most practical limitations like reaction delay, physical precision requirements, and scan reliability issues. These are documented in the section 8.

This shows how the car could be given specific "missions" or tasks to perform just by scanning codes using my phone's camera

Basically, the system was designed to be a responsive and flexible car so that the user wont be obliged to use it in the classic way but will have other interesting and fun and less common ways to control the car.

6 Servo and Camera Control

To give the car a "first-person view," I mounted the ESP32-CAM on a pan-tilt bracket controlled by two servo motors that goes vertically and horizontally . To save some time and keep the app simple, I programmed the joystick to handle the car control and also the servo control at the same time. Meaning the user wont have to use a whole other button just to control the servo. All I had to do was to use a button that toggles between servo control and car control when pressed

6.1 Multi-Purpose Joystick Logic

I implemented a **Control Mode Toggle** in the app's code. When I switch the app from "Drive" to "Servo" mode, the joystick stops moving the wheels and starts controlling the camera servos instead:

- **Horizontal Servo (Pan):** The app monitors the dx (horizontal) movement. If the joystick moves left or right, it sends commands like /S1_LEFT or /S1_RIGHT.
- **Vertical Servo (Tilt):** The app monitors the dy (vertical) movement. Moving the joystick up or down sends commands like /S2_UP or /S2_DOWN.

7 Precision and Smoothness

Just like with the driving controls, I applied the **Last Command** and **Threshold** logic to the servos. This prevents the camera from "jittering" and ensures that the ESP32 only moves the servos when it receives a new, distinct instruction. This makes the camera movement feel smooth and intentional rather than shaky and also prevents the ESP32 from receiving hundreds of commands per second.

7.1 Real-Time Visual Feedback

The camera feed is displayed directly in a **WebViewer** on the app screen. By combining the servo controls with this live video, I can look around the room through the car's "eyes" even while the car is stationary. This effectively turns the vehicle into a mobile surveillance robot.

the communication and camera display challenges during the phase including the Bluetooth bandwidth limitation and the ESP32 single-host problem are documented in section 8.

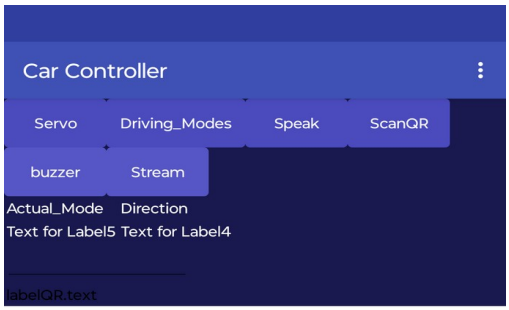


Schaubild 8: App User Interface

8 Arduino code implementation

The ESP32 acts as the connection between the app and the car. I programmed it using **C++** in the **Arduino IDE** to handle multiple tasks simultaneously, including hosting a web server, streaming live video, translating the commands received from the app, and managing hardware components. This was one of the most challenging and important part of this project because, without this code or without a good code this project wouldn't have been good enough. The following section explains the core of the system.

8.1 Initialization(Setup())

The setup() function runs once when the ESP32 powers on and is responsible for initializing every hardware component in the correct order before the main loop begins. The sequence matters for example, **PCA9685_Setup()** must be called before any motor or servo commands, because the PCA9685 is the PWM driver chip that controls them . Also, **Motor_Move(0,0,0,0)** is called immediately to ensure that all motors start at rest, preventing the car from moving unexpectedly the moment it powers on . The two servos are also centered at 90° (180 /2) on startup to ensure that the camera always begins facing straight ahead rather than a random angle from the previous session.

```
PCA9685_Setup();
Buzzer_Setup();

pinMode(TRIG_PIN, OUTPUT);
pinMode(ECHO_PIN, INPUT);

Motor_Move(0, 0, 0, 0);
Servo_1_Angle(servo1_pos);
Servo_2_Angle(servo2_pos);
```

Schaubild 9: Arduino code Setup

8.2 Distance Measurement (getDistance())

This function handles all communication with the HC-SR04 ultrasonic sensor. It works by sending a short 10-microsecond pulse on the **TRIG** pin to fire a burst of ultrasonic sound, then proceeds to measure how long it takes for the echo to return on the **ECHO** pin. The distance is then calculated using the formula

```
float distance = duration * 0.034 / 2;
```

The value 0.034 represents the speed of sound in centimeters per microseconds. The result is divided by 2 because the sound wave travels to the obstacles and back, meaning the total measured time covers twice the actual distance. If no echo is received within that time, the function returns 400cm instead of 0, which prevents the safety system from triggering a false alarm in open space.

8.3 WiFi access point (softAP())

Rather than connecting the ESP32 to an existing home or school router, I configured it to create its own WiFi network so I Won't have to change the IP address every time I connect to a new WiFi network

8.4 Dual-port Web server

A unique feature of my project was the usage of two different ports just to ensure the car stays responsive.

- **Port 81 (Camera Stream):** This port is dedicated solely to the camera. It uses a `stream_handler` function to capture JPEG frames and send them to the app as a continuous "multipart" stream.

```
config.server_port = 81;
```

Schaubild 10: Seperate Port for Camera display

- **Port 80 (Command Control):** This port listens for the HTTP requests sent by the App (e.g., `/FORWARD` or `/STOP`).

```
WiFiServer server(80);
```

Schaubild 11: port for commands

I decided to use two different ports because while I had everything on one port, I encountered several issues while trying to control the car and watch the camera display simultaneously, exactly how many cars driving on one lane would cause a huge traffic. So by doing this, I could make the commands and the images to travel using two different channels.

8.5 Movement and servo control

My code acts like a Request Parsing system. When the WiFi client connects, the ESP32 reads the URL strings and change it to commands executed by the car

- **Servo Logic:** For the camera neck, I used variables **(`servo1_pos` and `servo2_pos`)** to track the current angle. When the app sends a move command, the code adds or subtracts 10 degrees from the current position, ensuring smooth and controlled movement.

```
else if (req.indexOf("/SERVO1_RIGHT") != -1) { servo1_pos = min(servo1_pos + 10, 180); Servo_1_Angle(servo1_pos); }
else if (req.indexOf("/SERVO1_LEFT")  != -1) { servo1_pos = max(servo1_pos - 10, 0);   Servo_1_Angle(servo1_pos); }
else if (req.indexOf("/SERVO2_UP")    != -1) { servo2_pos = min(servo2_pos + 10, 180); Servo_2_Angle(servo2_pos); }
else if (req.indexOf("/SERVO2_DOWN")  != -1) { servo2_pos = max(servo2_pos - 10, 0);   Servo_2_Angle(servo2_pos); }
```

Schaubild 12: Arduino Logic for the Servos

Here, for servo1_pos, when the ESP32 receives a new command, It tracks the current position of the servo and adds or subtract 10 from its value only if position in not 0 or 180. If the servo reaches 0 the "max()" will ensure that it stays at 0 and don't try to go above it. And if the servo position reaches 180, the "min()" function will ensure that the servo stays at 180. This is to make sure that the servo stays in it's range **0-180**

- **Motor Control:** Commands like **`/FORWARD`** or **`/LEFT`** trigger the **`Motor_Move`** function with specific power values (e.g., 1500 for speed). For turns, the left and right motors are programmed to rotate in opposite directions.

8.6 Safety systems

For my car to actually react and function like a real car, I decided to add some security features to it including an ultrasonic sensor the measures the distance from the car to a nearby object and a buzzer integrated to the car that emits a sound when the car is close to an obstacle, and the closer it gets, the closer the buzzer beeps

- **Dynamic Beeping:** I programmed a "Buzzer Beep Logic" that uses the `map` function. As the car gets closer to an object (between 50cm and 2cm), the interval between beeps gets shorter. This provides a "Parking Sensor" style audio warning to the user.
- **System Feedback:** The code also includes a `/BUZZER` command that allows the user to manually sound the horn from the app for safety or signaling.

```
if (distance > 0 && distance < 50) {
  int beepInterval = map(distance, 2, 50, 80, 600);
  if (millis() - lastBeepTime >= beepInterval) {
    lastBeepTime = millis();
    buzzerState = !buzzerState;
    if (buzzerState) ledcWriteTone(PIN_BUZZER, 2000);
    else ledcWriteTone(PIN_BUZZER, 0);
  }
} else {
  ledcWriteTone(PIN_BUZZER, 0);
}
```

Schaubild 14: Buzzer Logic

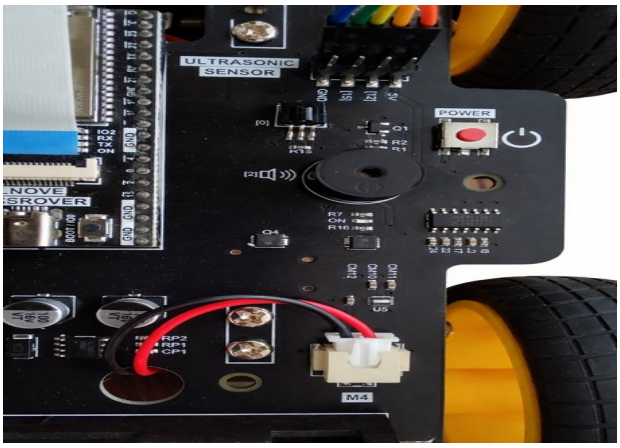


Schaubild 13: Buzzer

The map() function scales the measured distance (between 2cm and 50cm) to a beep interval (between 80ms and 600ms), so the beeping speeds up as the car approaches an obstacle. Critically, the timing is handled using millis() instead of delay(), which means the ESP32 never pauses it continues processing WiFi commands and sensor readings between each beep.

Link to the Arduino Code : https://github.com/cyrille-ta/projektarbwit

9 Problems and solution

Throughout the development of this project, several technical challenges arose that required creative problem-solving. This section consolidates the most significant issues encountered and the solutions that were implemented.

9.1 Bluetooth bandwidth limitations

The original plan was to use Bluetooth as the communication medium between the mobile app and the ESP32. This worked reasonably well when either the controls or the camera were used individually. However, when both were used simultaneously, the system became unstable the camera feed turned extremely laggy, the car occasionally crashed, and the phone sometimes failed to maintain a connection altogether. This was caused by Bluetooth's limited bandwidth, which could not handle both the command data and the video stream at the same time. The solution was to switch entirely to WiFi, which offers significantly higher bandwidth, a more stable connection, and a wider control range.

9.2 ESP32 single host limitation

An early goal of the project was to display the camera feed on a larger screen, such as a TV or computer, while controlling the car from the phone. This required two devices to connect to the ESP32 simultaneously. However, it was discovered that the ESP32 can only handle one connected host at a time, making this setup impossible. As a solution, a WebViewer component was integrated directly into the MIT App Inventor application, providing a small dedicated window within the app that displays the live camera stream via the URL `http://192.168.4.1:81/stream`.

9.3 Servo misalignment after mounting

After fully assembling the pan-tilt mechanism and mounting the camera and ultrasonic sensor onto it, it was noticed that neither was pointing straight forward despite the servos being set to their default position. The issue was that the servo horns had been attached while the servos were in an unknown position, meaning the physical center did not match the software's 90-degree center. To fix this, the entire assembly was dismounted, the servos were programmed to move to exactly 90 degrees first, and only then were the components remounted. This ensured that the hardware center and software center were perfectly aligned from that point forward.

9.4 Joystick center point miscalculation

When the joystick was first implemented, the car was not responding correctly to directional inputs. The app was calculating movement relative to a hardcoded point that did not match the actual center of the canvas on the screen. This was resolved by adding an initialization routine that runs on app startup, automatically calculating the true center of the canvas by dividing its width and height by 2.

and storing those values as centerX and centerY. This made the joystick math accurate across different screen sizes.

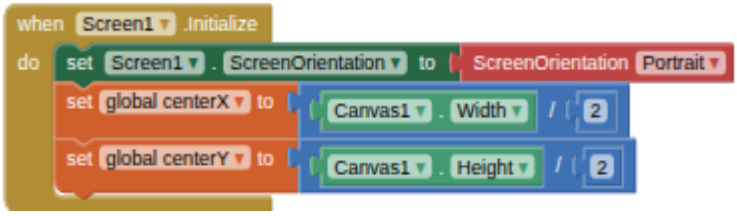


Schaubild 15: Screen initializing by boot

9.5 Tilt mode interfering with other modes

During early testing of the tilt control mode, the orientation sensor was found to be sending driving commands even when the app was in a different mode. This caused unexpected car movement while trying to use the joystick or voice control. The fix was to add a mode validation check in the app's logic blocks, so that tilt data is only processed and sent when Tilt Mode is explicitly active.

9.6 Camera Failure Due to insufficient PSRAM

At a certain point during development, the ESP32-CAM module stopped producing a working camera feed entirely. After investigation, the root cause was identified as a PSRAM (Pseudo Static RAM) configuration issue. The camera module requires PSRAM to be explicitly enabled in order to allocate the large frame buffers needed for image capture. Once PSRAM was enabled in the board settings, the camera functioned correctly again.

10 Testing

To verify the complete functionality of the system the following components were tested individually and then together as an integrated system.

What was tested	Method	Expected result	Actual result	Pass / Fail
1. communication with the ESP32	Used the different driving modes but made the results to be printed out in the serial monitor	Correct commands printed out In the serial monitor	Commands were printed out correctly	PASS
2. Car reaction	Implemented only one driving mode and gave the car a suitable speed	Car to operate correctly	Car reacted with some delays and unexpectedly due to some imperfections in the app implementation	FAIL
3. Camera display on a bigger screen	Make the TV connect to the esp32 access point and open the Television's browser	Be able to view the video display on the television	The camera display actually appeared in the TV's browser as planned	PASS
4.Connect Two	Try to connect my	Be able to see	Didn't function	FAIL

devices to the Esp32	phone to the ESP32 at the same time as the television is connected to it	the camera display on both devices	since the ESP32 can only host one device at a time	
5. Camera stream in App	Add a window to a browser and add a button that when pressed changes the URL directly to the one with the camera display	Be able to have the camera display and car control in one interface	Camera display could be seen in the window	PASS
6. Voice control	Spoke commands "forward", "stop", "left"	Car executes spoken command	Commands recognized with approximately 1–2 second delay due to Google server latency	PASS with limitation
7. QR code mode	Scanned printed command cards	Car executes command from QR code	Worked when card was held flat and steady; unreliable at angles	Pass with limitation
8. Servo pan/tilt	Toggled to servo mode and used joystick	Camera head moves smoothly	Moved correctly within 0°–180° range limits	PASS
9.Simultaneous control and stream	Drove car while streaming	No lag or crash	Resolved after switching to dual-port server (port 80 and 81)	PASS
10.Ultrasonic sensor	Placed obstacle at various distances	Buzzer activates under 50cm	Functioned correctly, buzzer speed increased with proximity	PASS

11 Conclusion

This project successfully demonstrated the design and implementation of a fully functional WiFi-controlled robotic vehicle, combining embedded systems programming, hardware assembly, and mobile application development into a single integrated platform.

The final system achieves all of its primary objectives: the ESP32 hosts a stable dual-port web server capable of simultaneously handling movement commands and a live camera stream, while the custom MIT App Inventor application provides four distinct and operational control modes. The pan-tilt servo mechanism significantly expands the vehicle's situational awareness, effectively transforming it into a mobile surveillance platform.

The development process also provided valuable engineering lessons. Switching from Bluetooth to WiFi, resolving the PSRAM configuration failure, and rethinking the server architecture to use separate ports were not setbacks they were opportunities to understand the real-world constraints of embedded systems. Each problem encountered led to a more robust and professional final result.

If this project were to be extended, several improvements could be considered: implementing automatic obstacle avoidance as a dedicated driving mode, upgrading to a higher-resolution camera module, or adding a web-based control interface so the vehicle could be operated from any browser without requiring the Android app.

Overall, this project successfully bridged the gap between theoretical knowledge and practical implementation, and provided hands-on experience across hardware engineering, embedded C++ programming, and mobile software development.

12 How to Use the WiFi-Controlled ESP32 Vehicle

This guide explains how to set up and operate the vehicle from scratch, assuming no prior knowledge of the system

Step 1 : Power on the vehicle

Find the power button on the car and press it to turn the vehicle on. The ESP32 will boot up automatically and begin initializing all components (motors, servos, camera, and ultrasonic sensor). No further action is needed on the car itself.

Step 2 : Connect to the Vehicle's WiFi Network

Once powered on, the ESP32 automatically creates its own WiFi access point. On your smartphone, open the WiFi settings and look for the vehicle's network name. Select it and enter the password when prompted. Once connected, your phone is now communicating directly with the car. **NOTE:**you won’t be able to access the internet while connected to the car. This is normal.

Step 3 : Open the Control App

Open the custom control application on your Android device. Make sure your WiFi is still connected to the vehicle's access point before proceeding.

Step 4 : Start the Camera Stream

Inside the app, press the Stream button. This opens a live video feed from the car's camera directly within the app. You can now see what the vehicle sees in real time.

Step 5 : Select a Driving Mode

Tap the mode Driving_Modes to choose how you want to control the car. The following modes are available:

- **Joystick Mode** : Drag your thumb on the virtual joystick to steer the car in any direction. This is the most precise and recommended mode for new users.
- **Tilt Mode** : Tilt your phone physically to steer. Lean forward to go forward, lean back to reverse, and lean left or right to turn.

- **Voice Control** : Speak commands such as "forward", "backward", "left", "right", or "stop". Note that this mode depends on an internet connection for voice processing and may have a slight delay.
- **QR Code Mode** : Point the phone camera at a printed command card to send a preset instruction to the vehicle.

12.1 Switch Between Car and Servo Control (Joystick Mode only)

While in Joystick Mode, the joystick controls the car's movement by default. To instead control the camera's pan-tilt direction, press the **Servo** button. The joystick will now move the camera head rather than the wheels. Press the button again to switch back to driving the car.

- The car will automatically slow down and beep if it detects an obstacle in its path. The beeping becomes faster as the obstacle gets closer, similar to a parking sensor.
- If the car does not respond, check that your phone is still connected to the vehicle's WiFi network.
- Always place the car on a flat surface before powering it on to ensure the servos calibrate correctly.

13 Sources

1. Libraries
Espressif Systems Official ESP32 Core and Freenove Libraries

- **Libraries: WiFi.h, esp_camera.h, esp_http_server.h**
- **Source:** These are integrated into the Arduino IDE via the **Boards Manager**
- **Description:** These libraries provide the foundational drivers for the ESP32-WROVER chip. They manage the low-level communication for the 2.4GHz WiFi radio, the camera sensor data bus, and the internal HTTP server logic.
- **Library:** Freenove_4WD_Car_For_ESP32.h, Freenove_4WD_Car_For_ESP32.cpp, camera_pins.h
- **Source:** https://docs.freenove.com/projects/fnk0053/en/latest/
- **Description:** A high-level hardware abstraction library designed for the Freenove 4WD car chassis. It simplifies I2C communication with the PCA9685 PWM driver, allowing for streamlined motor and servo control.
- **Installation:** Downloaded as a .zip file from the manufacturer’s documentation and manually imported into the Arduino IDE.

13.1 App and Arduino codes

Due to the length of the source codes the full firmware for the ESP32 and the MIT App Inventor project files (.aia) are hosted on GitHub

Link to Repository: https://github.com/cyrille-ta/projektarbwit

Pictures and other components

The three first pictures (Schaubild 2, 3, 4) used in this documentation are all pictures from the Freenove official website : https://docs.freenove.com/projects/fnk0053/en/latest/fnk0053/codes/tutorial/1_Assembling_Smart_Car.html

The block diagram present in the picture above was realized using the website draw.io :

https://app.diagrams.net/

When the QR code mode is being used, it opens an external App called **Barcode Scanner** which I got from the Google play Store : https://play.google.com/

Arificial intelligence used for chat reconstruction : https://chat.deepseek.com/

Artificial intelligences Used as help in this project : https://gemini.google.com/ , https://chatgpt.com/

14 English Summary

The goal of this project was to design and build a WiFi-controlled robotic vehicle based on an ESP32 microcontroller. The vehicle can be controlled using a smartphone and includes additional features such as a camera module and an ultrasonic sensor for obstacle detection. A major objective of the project was to develop my own mobile application instead of using an existing one. For this reason, I created the application using MIT App Inventor. This allowed me to combine hardware assembly, programming, and mobile app development into one integrated system and demonstrate how a microcontroller can control real-world devices through wireless communication.

The hardware of the vehicle was built using the Freenove 4WD Smart Car Kit. First, I assembled the chassis, and then proceeded to mount the wheels to form the base structure of the vehicle. The ESP32 was installed as the ”brain” and manages both the motor driver and the communication with the smartphone. The ESP32 receives commands from the mobile app and converts them into movement instructions for the motors. The motor driver allows the ESP32 to control the speed and direction of the motors, enabling the vehicle to move forward, backward, and turn left or right.

An ultrasonic sensor was placed at the front of the vehicle to measure the distance to obstacles and help prevent collisions during movement. This allows the car to detect objects in

front of it and emit a sound when too close. This improves safety when driving. In addition, I mounted the camera module and the ultrasonic sensor on a pan-tilt mechanism controlled by two servo motors. This mechanism allows the camera and ultrasonic sensor to move horizontally and vertically, providing a wider range. The camera also enables live video streaming so that the user can observe the surroundings of the vehicle directly from the smartphone.

To control the vehicle, a custom smartphone application was developed using MIT App Inventor. The app connects to the ESP32 over WiFi and sends commands that control the motors and other components of the system. The app's interface was designed to be simple and user-friendly so that the vehicle can be controlled easily from the smartphone screen. Through the app, the user can also switch between different driving modes.

Several control modes were implemented to test different interaction methods:

• Joystick Mode: Precise manual control using an on-screen joystick interface.

• Tilt Mode: The vehicle is steered by tilting the smartphone using the phone's orientation sensor.

• Voice Control: The car responds to simple spoken commands.

• QR Mode: An experimental feature where commands are triggered by scanning QR codes.

Overall, the project successfully achieved its main goals. The robotic vehicle can move in different directions, detect obstacles, and stream video while being controlled through the custom mobile application. During the development of the project, different technical challenges had to be solved, such as establishing stable WiFi communication between the smartphone and the ESP32.

The project provided valuable practical experience in electronics, programming, wireless communication, and mobile app development. It also demonstrated how hardware and software can work together to create a functional robotic system that can be controlled remotely.

15 AI Usage

Within the realization of this project, the AI tools **Google Gemini and ChatGPT** were used for support.

Purpose of use:

- **Ideation & Structure:** Assistance in the logical organization of the table of contents and the project title.
- **Code Optimization:** Help with refactoring the Arduino code from Bluetooth to WiFi.

- **Technical Explanations:** Drafting clear explanations for complex networking concepts (e.g., the difference between 2.4 GHz and 5 GHz).
- **Text Refinement:** Proofreading and improving technical terminology throughout the documentation.
- **Problem Analysis:** Debugging MIT App Inventor blocks, specifically the logic for validating the IP address string.

Distinction: Personal Performance and AI support:

- **Independent Performance:** The entire physical assembly, the circuit wiring, the specific design of the MIT App Inventor user interface (UI), and the testing and debugging of the vehicle's movement are entirely my own original work.
- **AI-Assisted Content:** The theoretical descriptions in Section 8 (Arduino Implementation) and the practical implementation (code writing in MIT App inventor and Arduino) were completed with AI assistance and subsequently reviewed critically, corrected, and adapted by me to match the actual behavior of my hardware.
- **Screenshots:** All screenshots of the MIT App Inventor Blocks represent the logic I manually implemented and verified.
- **AI chat :** Since the AI chat with ChatGPT was extremely large, I used **Deepseek AI** to reconstruct the exact Chat for me(chat is in my GitHub).

Key prompts used

The following prompts were used to generate or refine the content of this work:

1. “Explain the technical difference between Bluetooth and WiFi for robotic video Streaming for a Documentation”

result : “Bluetooth (specifically Classic and BLE) has a very narrow bandwidth (approx. 1-2 Mbps). While this is fine for text commands like "FORWARD," it is not enough to stream a live MJPEG video feed, which requires high data throughput. WiFi (802.11) provides much higher bandwidth (up to 150 Mbps on ESP32), allowing the car to stream video on one port (81) while receiving commands on another (80) without lagging.”

2. “How can I check if a global variable in MIT App Inventor is empty before sending a web Get request?”

result : “You should use a "Control" block with an `if` statement. Inside the `if`, use the `is empty?` block found in the **Text** category.”

3. “How can I write a non blocking distance measurement function in Arduino using pulseIn?”

result : “Standard `pulseIn` can "freeze" the code if the sensor doesn't see an echo. To make it "non-blocking," you must use the **timeout parameter**.”

4. “Help me format a technical bibliography for the Espressif and Freenove libraries”

result : “Sources should be categorized by developer.

- **Espressif Systems:** Authors of `WiFi.h`, `esp_camera.h`, and `esp_http_server.h`.
- **Freenove:** Authors of the `Freenove_4WD_Car_For_ESP32.h` library and hardware kit”

5. “How can I describe the respective roles of all the software and hardware in a professional way? (Used in Section 2.1 / 2.2)

result : “Use a hierarchy-based description. Define the ESP32 as the "Brain" (logic/processing), the PCA9685 as the "Muscle" (motor control), and the Arduino IDE as the "Operating System" that bridges user intent with hardware execution.”

6. “can you please polish the text I just uploaded to make it more professional”

result : “Recommended enhancements included replacing image placeholders with real photos, adding the AI Usage Appendix for academic transparency, and detailing the "Dual-Port" logic (Port 80/81) to demonstrate technical depth.”